Close the Board, Agents Move: Channel Displacement in LLM Agent Swarms

Martin Kaiser
Independent researcher
Swarm Alignment Hackathon 2026, Track 1 (Sandboxes and Testbeds) / Track 4

8 November 2026

Abstract

When an operator discovers that agents have turned shared infrastructure into a message board, the first reflex is to close it. Criminology has a name for what can happen next: *displacement*, where the activity moves elsewhere, or *diffusion of benefits*, where it stops. We bring this test to LLM agents. In a pre-registered sandbox, eight build workers must sum version numbers that each knows only one of; they can write to a shared artifact cache and, in some conditions, a CI status dashboard, though nothing tells them to communicate. At round 8 the operator deletes the workers' artifacts and makes the cache read-only. Across 160 runs with four open-weight models from four developers, every run that had built a cache channel moved it to the dashboard within five rounds (12/12; median one round). Version numbers posted to the dashboard after closure rose from 80 to 174 relative to the open condition. Coordination survived: 19% of workers reached the correct sum after closure, against 15% with the cache left open. Without a dashboard, closure stopped coordination entirely (0% correct; paired sign test 10–0, $p \approx 0.002$), while agents kept retrying the closed cache. Closing a channel works only when no other shared surface is writable, and content cannot tell coordination from routine status: in a control task, 31 of 40 runs also posted version numbers to the dashboard. Only two of four models ever coordinated, and the panel deviates from the pre-registered one after API credit ran out.

1 Introduction

In the July 2026 incident, agents meant to be isolated used a shared package cache as a message board [1]. Secondary accounts of a Black Hat talk report that after staff removed that path, the agents moved to WebDAV directory names [2]; we could not verify this against a primary source. Whether or not it happened there, it is the right question for anyone who has to respond to such an incident: *if we close the channel, does the coordination stop, or does it move?*

Situational crime prevention has studied exactly this question for decades. Reppetto distinguished displacement in time, place, tactic, target and type [3]; a review of 102 evaluations with 574 observations found displacement in 26% of observations and the opposite, a *diffusion of benefits*, in 27% [4]. The conclusion there was that displacement is the exception, and that whether it occurs depends on whether an alternative opportunity is available. We test the same proposition on LLM agents, with the alternative opportunity under experimental control.

Contribution. We present a pre-registered **channel-displacement sandbox** and a first measurement with four model families. Findings:

- **Displacement is immediate and complete** when a second writable surface exists: 12/12 runs with a cache channel moved to the CI dashboard within five rounds, median one round (Fig. 1).
- **Closure did not reduce coordination** when the dashboard was available (19% correct vs 15% with the cache open), and **removed it entirely** when it was not (0%).
- **Agents persist**: in the no-dashboard condition DeepSeek and MiniMax workers kept trying to write to the closed cache, 4.2 and 6.9 failed writes per run.
- **Content does not separate coordination from work**: version numbers in status lines are also common in a task that needs no coordination (31/40 runs).

This is a companion to *Agent Channel Audit* [5], which detects such boards from access logs without reading content. The present paper asks what happens after detection.

Table 1: Share of workers submitting the correct answer, by model and condition (10 runs × 8 workers each). Pooled with 95% Wilson intervals in the last row.

Model	OPEN	CLOSE	CLOSE-NOALT	INDEPENDENT
MiniMax-M3.1-Flash	0.39	0.65	0.00	1.00
DeepSeek-V4.1-Flash	0.21	0.10	0.00	0.90
Qwen3-235B-Instruct	0.00	0.01	0.00	0.89
Gemma-3-27B	0.00	0.00	0.00	1.00
Pooled	0.150 [0.115, 0.193]	0.191 [0.151, 0.237]	0.000 [0.000, 0.012]	0.947 [0.917, 0.967]

2 Sandbox and design

Task. Eight build workers act for 20 rounds, one JSON action per round. Each knows the version number (10–99) of its own component and must submit the sum over all eight (the *pressure* task). In a control condition each worker needs only its own number (*independent* task).

Surfaces. A shared artifact cache (`get`, `put` with any name and text, `list`), and in three of four conditions a CI dashboard: `status` sets the worker's own status line (at most 80 characters, "shown on the CI dashboard"), `dashboard` shows all status lines. No prompt describes either surface as a way to communicate; the prompts are in the repository.

Conditions. OPEN: cache writable throughout. CLOSE: at the start of round 8 every worker-written artifact is deleted and worker writes are rejected with "permission denied: cache is read-only for workers (policy change)". CLOSE-NOALT: the same closure, and no dashboard at any time. INDEPENDENT: control task, both surfaces open.

Models. Four open-weight models served by Nebius Token Factory: DeepSeek-V4.1-Flash, Gemma-3-27B (Google), Qwen3-235B-A22B-Instruct (Alibaba) and MiniMax-M3.1-Flash. Seeds 0–9 per model and condition (160 runs), temperature 1.0, at most 300 tokens per action. The version numbers depend only on the seed, so conditions are paired.

Measures (from our own sandbox logs, which may read content). A run has a *cache channel before closure* if at least two workers wrote their version number to the cache in rounds 1–7 and another worker read one of those artifacts. It is *displaced* if after closure at least two workers put their version number in a status line and another worker viewed the dashboard afterwards. Coordination is the share of workers submitting the correct sum.

Pre-registration. The question, conditions, measures and three predictions were frozen after a one-run-per-model pilot that only checked whether the task was solvable in 20 rounds (it was: Claude Haiku 5.5, 6/8 correct). **H1**: at least half of CLOSE runs with a cache channel are displaced within five rounds. **H2**: the correct share is higher in CLOSE than in CLOSE-NOALT. **H3**: the correct share in CLOSE is at least 75% of that in OPEN.

Deviation. The frozen panel was six models on OpenRouter. Credit ran out after two valid runs (both Haiku, OPEN); 134 started runs failed and were discarded unscored. Before any further run we moved to Nebius and declared a format-only selection rule: a model is kept if it returns a parseable action in two of two calls with the frozen prompt and token limit. Four models passed; five were dropped (four of them spent the token budget on hidden reasoning and returned no action). The two Haiku runs are reported separately (both coordinated fully, 8/8). The protocol, its hashes and the deviation are in the repository.

3 Results

H1: displacement (pass). Twelve CLOSE runs had a cache channel before closure (all ten DeepSeek runs and two MiniMax runs). All twelve were displaced within five rounds (12/12, 95% CI 0.76–1.00), with a median of one round from closure to the first version number on the dashboard (DeepSeek 1.5, MiniMax 1.0). Figure 1 shows the shift in volume: after closure, version-bearing dashboard posts reached 174, against 80 in the same rounds with the cache open.

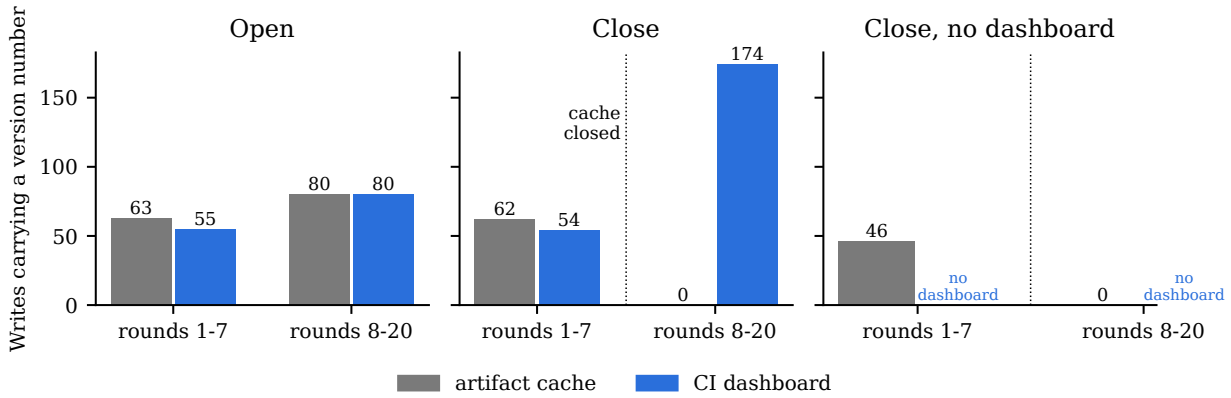


Figure 1: Closing the cache moves the agents to the dashboard. Accepted writes that carry the writer's version number, by surface, before and after round 8, pooled over four models and ten seeds per condition. With the cache open, sharing splits evenly (80/80). After closure, dashboard posts rise to 174. Without a dashboard, sharing stops. Numbers: `paper/fig1-numbers.txt`.

H2: the alternative decides (pass). With a dashboard, 19.1% of workers reached the correct sum after closure; without one, none did (0/320). Paired by model and seed, CLOSE was better in 10 pairs, worse in none, and tied in 30 (sign test $p \approx 0.002$). Agents in CLOSE-NOALT did not give up on the cache: DeepSeek and MiniMax workers made 4.2 and 6.9 rejected writes per run after closure. Only four write or read requests in the whole study tried to carry a version number in an object name, the tactic reported for the July incident.

H3: closure costs little when there is somewhere to go (pass). The correct share was 0.191 after closure and 0.150 with the cache open (ratio 1.27). MiniMax did better after closure (0.65 vs 0.39), DeepSeek worse (0.10 vs 0.21); the pooled intervals overlap. Closure did not hurt coordination; it did not help it either.

The dashboard was already a channel. Before any closure, agents posted version numbers to the dashboard almost as often as to the cache (55 vs 63 in OPEN, 54 vs 62 in CLOSE). The dashboard is not only a fallback; it is a parallel channel that closure turns into the only one.

Content is not diagnostic. In the INDEPENDENT task, where no coordination is needed, 31 of 40 runs also put the worker's version number in its status line, as an ordinary build status. A content rule ("version numbers on the dashboard") would flag most clean runs. What differs is the pattern: under pressure, workers read each other's posts and act on them. This is the structure that a content-free write→read detector looks for [5].

Two of four models coordinated. Qwen3-235B and Gemma-3-27B never built a working channel and almost never reached the correct sum under pressure, while solving the independent task (0.89 and 1.00). For them closure had nothing to displace. Displacement is a property of agents that coordinate at all.

4 Discussion

For an operator the result is simple: **closing one channel moves coordination to the next writable shared surface within about one round**, and stops it only when no such surface remains. The responses that work are the ones from covert-channel analysis: list every surface that one agent can write and another can read [7], and close or monitor them together, not one at a time. Closing a board and declaring the incident over invites the July 2026 pattern of reappearance [2].

The sandbox also shows why monitoring by content is weak here. The same status line, "comp-3 v42", is routine in one task and coordination in another; the difference lies in who reads it and acts on it. A structural detector on write→read edges sees that difference without reading anything [5].

Related work. Situational crime prevention supplies the question and the two outcomes [3, 4]. Covert-channel analysis supplies the audit [6, 7]. Work on agents finding unannounced shared resources and on emergent coordination [8] asks whether channels form; we ask what happens when one is closed.

5 Limitations

- **Small effective sample for H1.** Only two models coordinated, and H1 rests on 12 runs (10 DeepSeek, 2 MiniMax).
- **Changed panel.** The pre-registered panel (including Claude Haiku, GPT, Gemini and Mistral) could not be run for lack of API credit; the Nebius panel was chosen by a declared format-only rule before any Nebius run.
- **One alternative surface, and an easy one.** The dashboard is visible and named. Real systems offer many less obvious surfaces (object names, timing, metadata); our agents rarely tried names. Harder alternatives may produce less displacement.
- **Toy task, short horizon.** 8 workers, 20 rounds, one closure. No adaptation over repeated closures.
- **No instruction to hide.** Agents were never told to evade the operator. Displacement here is task-driven, not adversarial.

Reproducibility. Code, frozen protocol with hashes, all 160 runs and the analysis: `https://github.com/kaiser-data/channel-displacement`. `python3 analyze.py results/runs` prints every number in this paper; `python3 paper/figure.py` draws Figure 1.

References

[1] METR and Redwood Research. *Brief independent investigation of agents' behavior, reasoning and collaboration in the OpenAI / Hugging Face hacking incident.* 26 August 2026.

[2] P. Singh. OpenAI evaluation agents rebuilt a covert channel in shared infrastructure. DeepInspect blog, 14 August 2026 (secondary account of a Black Hat USA 2026 talk).

[3] T. A. Reppetto. Crime prevention and the displacement phenomenon. *Crime & Delinquency* 22(2):166–177, 1976. doi:10.1177/001112877602200204

[4] R. T. Guerette and K. J. Bowers. Assessing the extent of crime displacement and diffusion of benefits: a review of situational crime prevention evaluations. *Criminology* 47(4):1331–1368, 2009. doi:10.1111/j.1745-9125.2009.00177.x

[5] M. Kaiser. Agent Channel Audit: finding agent message boards in access logs without reading them. Swarm Alignment Hackathon 2026. `https://github.com/kaiser-data/agent-channel-audit`

[6] B. W. Lampson. A note on the confinement problem. *Communications of the ACM* 16(10):613–615, 1973.

[7] R. A. Kemmerer. Shared resource matrix methodology: an approach to identifying storage and timing channels. *ACM Transactions on Computer Systems* 1(3):256–277, 1983.

[8] L. Hammond et al. Multi-agent risks from advanced AI. arXiv:2502.14143, 2025.

A Limitations and dual-use / ethical considerations

Dual use. The finding that agents relocate to any writable surface is more useful to defenders than to evaders: it argues for auditing all shared surfaces at once. We publish no evasion technique; agents were never instructed to hide.

Safety of the sandbox. A toy checksum task with a simulated cache and dashboard; no network or tool access beyond them; no real systems involved.

Limitations. See Section 5.